



Voyage Analytics

Travel Analytics Platform for Hotel Recommendations and Flight Price Prediction

PROJECT REPORT

Prepared by: Vinay Pandey

September 2026

1. Executive Summary

Voyage Analytics is an end-to-end travel analytics and MLOps project combining a hotel recommendation engine with a machine-learning-based flight price prediction service. The solution exposes a user-facing Streamlit dashboard and integrates the prediction model through a Flask REST API, MLflow model management, Docker containerization, Kubernetes deployment, and horizontal pod autoscaling. An Apache Airflow DAG validates data, trains the flight-price model, and verifies the resulting artifact.

The final deployed flight model is registered in MLflow as `Voyage_Flight_Price_Model` with the champion alias. The deployed artifact was verified as an sklearn Pipeline containing OneHotEncoder preprocessing and a LightGBM regressor. The production service was validated through the Flask health endpoint and prediction endpoint, and the Streamlit interface was validated for both hotel recommendations and flight price prediction.

2. Objectives

- Build a practical travel recommendation experience for selecting hotels by destination, duration, and budget.
- Develop a regression-based flight price prediction model using categorical, numerical, and date-derived features.
- Track and serve the production model through MLflow and its champion alias.
- Expose predictions through a Flask REST API.
- Containerize the inference service with Docker and deploy it on Kubernetes.
- Demonstrate horizontal scaling with Kubernetes HPA.
- Provide a clean Streamlit dashboard for demonstration and evaluation.
- Automate the model workflow using Apache Airflow and document the complete system.

3. Project Scope and Deliverables

Component	Purpose	Status
Hotel Recommendation	Destination, trip-duration, and budget-based recommendations	Validated
Flight Price Prediction	MLflow-managed regression inference	Validated
Flask REST API	/health and /predict endpoints	Validated
MLflow	Experiment/model registry and champion alias	Validated
Docker	Containerized Flask inference service	Validated
Kubernetes	Deployment, service, probes, and scaling	Validated
HPA	CPU-based horizontal scaling	Validated
Airflow	Validate → train → verify workflow	Validated
Streamlit	Interactive front-end for demonstration	Validated

4. System Architecture

The solution is organized as a layered workflow from data preparation to production inference:

Layer	Implementation
Data Layer	hotels.csv and flights.csv
Orchestration	Apache Airflow DAG: validate_data → train_model → verify_model
Experiment & Registry	MLflow tracking server + registered model + champion alias
Inference API	Flask application serving /health and /predict
Container Layer	Docker image travel-mlops-flask
Deployment Layer	Kubernetes Deployment + NodePort/Service + HPA
Presentation Layer	Streamlit dashboard

User → Streamlit → Flask API → MLflow Champion Model → Prediction
↳ **Kubernetes Deployment / HPA**

The solution is organized into distinct presentation, serving, model-management, automation, and deployment layers. This separation allows the Streamlit interface to remain independent of the production model artifact while the same MLflow-registered model can be served through a repeatable API deployment.

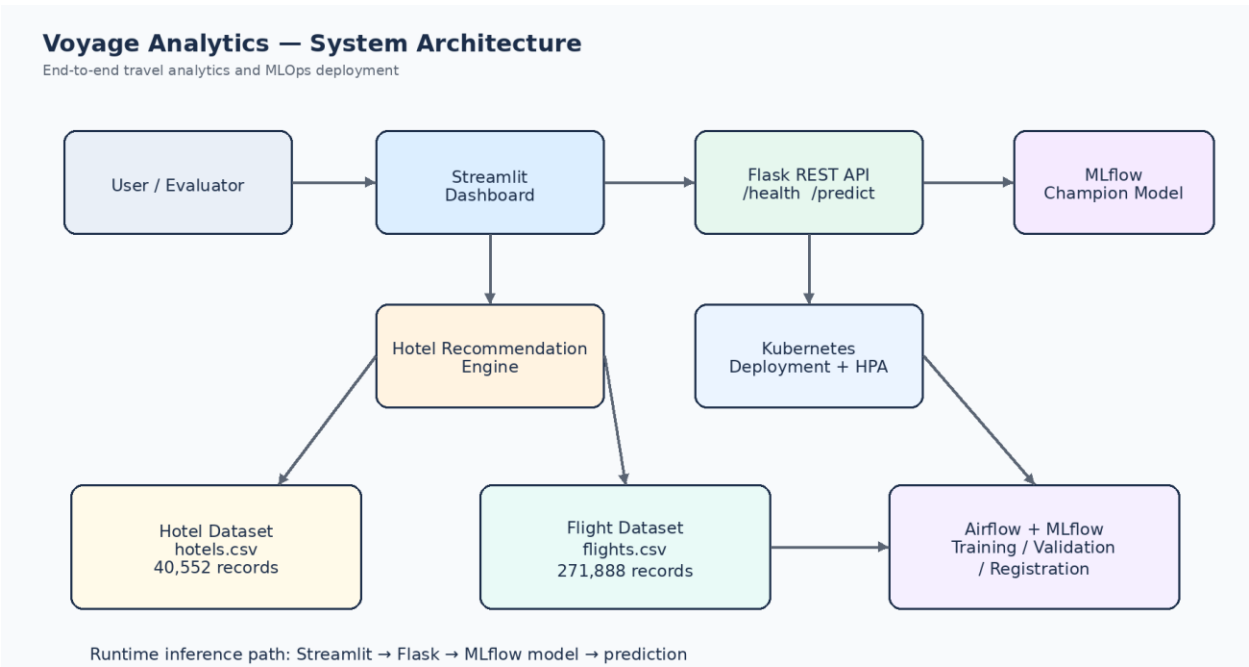
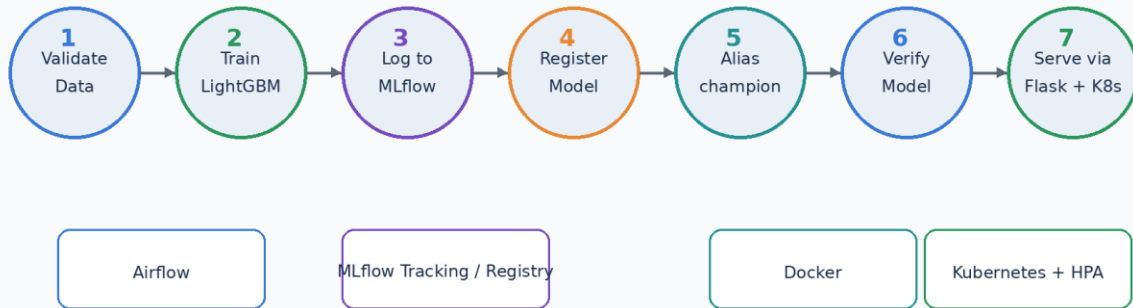


Figure 1. Overall system architecture and data flow.

The training path is orchestrated rather than executed manually. Airflow validates the data and coordinates training and verification; MLflow records the experiment and registered artifact; the production service loads the champion alias at runtime. This provides a clear lifecycle from dataset to deployable model.

Voyage Analytics — MLOps Lifecycle

Data validation, training, registry, verification, and serving



The same registered champion artifact is used for production inference, reducing training/serving drift.

Figure 2. MLOps lifecycle from data validation and training to model serving.

For inference, the browser communicates with Streamlit, which calls the Flask service through a local Kubernetes port-forward. The Kubernetes Service routes requests to healthy Flask replicas. Health probes protect traffic flow while HPA provides horizontal scaling when resource utilization increases.

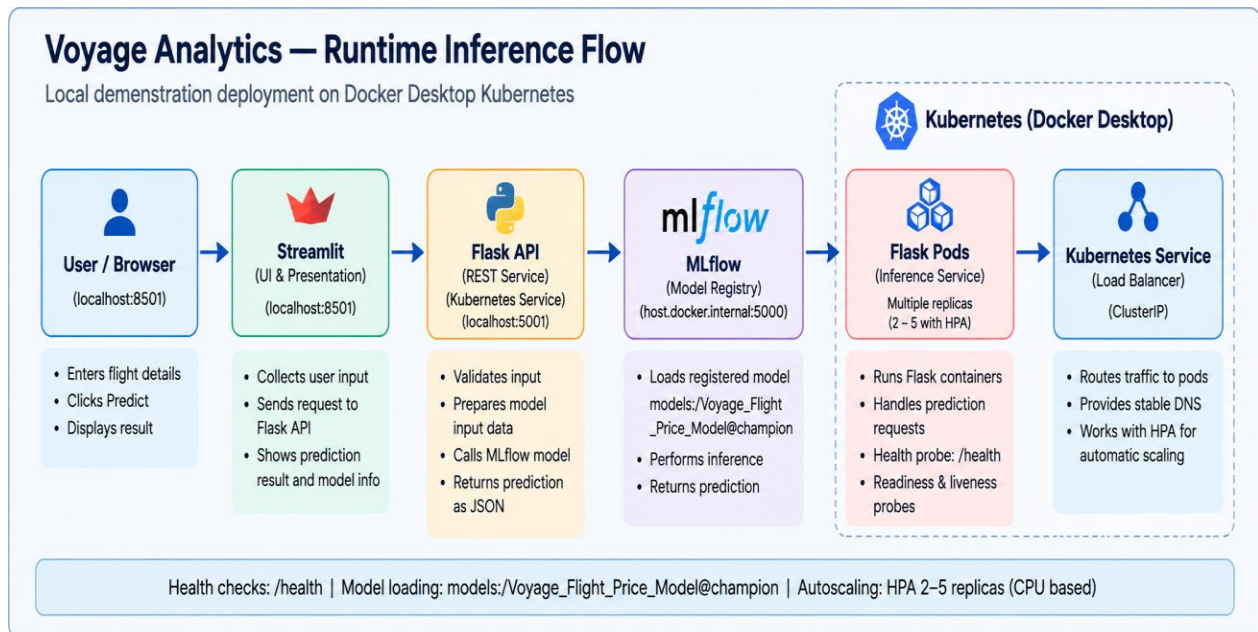


Figure 3. Runtime inference path on Docker Desktop Kubernetes.

5. Data Description

Architecture Design Notes

Presentation layer: Streamlit provides the user-facing hotel recommendation and flight price prediction workflows. Hotel recommendations are calculated locally from the hotel dataset, while flight prediction requests are sent to the Flask REST API.

Service layer: Flask exposes /health for Kubernetes and operational checks and /predict for inference. The API validates the request body, validates the date format, derives year/month/day/day_of_week features, constructs the model input DataFrame, and returns the prediction with model metadata.

Model-management layer: MLflow provides experiment tracking and the Model Registry. The deployed application loads models:/Voyage_Flight_Price_Model@champion, keeping the serving code independent from a fixed local model file.

Automation and deployment layer: Airflow coordinates the model lifecycle, Docker packages the Flask service, Kubernetes manages replicas and probes, and HPA can adjust the number of replicas within the configured bounds.

Resilience design: the API returns explicit HTTP 400 responses for invalid input and HTTP 500 responses for unexpected inference failures. Kubernetes health probes prevent unready pods from receiving traffic, while MLflow host-header configuration allows the containerized service to communicate safely with the local MLflow server.

5.1 Hotel Dataset

The Streamlit application reports 40,552 hotel records, 9 destinations, and 9 hotel names. Hotel recommendations are produced directly from the hotel dataset by filtering on destination and maximum total budget, then prioritizing the requested trip duration and lower total cost.

5.2 Flight Dataset

The Streamlit application reports 271,888 flight records, 9 origin cities, 9 destination cities, and 3 agencies. Flight features include origin, destination, flight type, time, distance, agency, and travel date. The application derives year, month, day, and day-of-week features before calling the model.

5.3 Currency

The project data and documentation use US dollars (\$) for travel prices. The final Streamlit UI therefore displays hotel and flight prices using the \$ symbol.

6. Flight Model Development

6.1 Feature Engineering

- Categorical variables: from, to, flightType, and agency.
- Numerical variables: time, distance, year, month, day, and day_of_week.
- Categorical encoding: OneHotEncoder(handle_unknown="ignore").
- The preprocessing pipeline is combined with the regression estimator so the same transformations are reused at inference time.

6.2 Model Selection

The project evaluated multiple regression approaches. The documented evaluation reports LightGBM with MAE $\approx$ \$23.81 and $R^2 \approx 0.9931$, while XGBoost achieved MAE $\approx$ \$23.35 and $R^2 \approx 0.9930$. The deployed champion artifact inspected during final validation is an sklearn Pipeline whose final estimator is a LightGBM regressor.

Model	MAE	R^2	Role
Linear Regression	250.13	0.2622	Baseline
Lasso Regression	250.13	0.2622	Regularized baseline
Ridge Regression	250.13	0.2625	Regularized baseline
Elastic Net	250.13	0.2622	Regularized baseline
LightGBM	23.81	0.9931	Deployed model family
XGBoost	23.35	0.9930	Best documented benchmark

7. MLOps Implementation

7.1 Airflow

The Airflow DAG `flight_price_ml_pipeline` provides the main workflow. The validated run contains three successful tasks: `validate_data`, `train_model`, and `verify_model`.

7.2 MLflow

- Registered model: `Voyage_Flight_Price_Model`
- Model version: 1
- Alias: `champion`
- Status: `READY`
- Production inference through `models:/Voyage_Flight_Price_Model@champion`
- Deployed artifact type: sklearn Pipeline with LightGBM regressor.

7.3 Flask API

The Flask application exposes `/health` for readiness/health validation and `/predict` for inference. The prediction endpoint validates required fields, parses the date, creates the expected model input structure, invokes the MLflow model, and returns the predicted price together with model name and alias.

7.4 Docker and Kubernetes

The Flask inference service runs in the `travel-mlops-flask` container image and is deployed as a Kubernetes Deployment. The application uses HTTP probes on `/health` for startup, readiness, and liveness. The Kubernetes Service exposes port 5001, and an HPA targets CPU utilization with a configured range of 2 to 5 replicas.

8. Validation and Testing

7.5 Production Readiness Design

Validation was performed at multiple system boundaries. The MLflow Registry was checked for the production model version and champion alias; Kubernetes rollout status and pod readiness were checked; Flask /health returned HTTP 200; Flask /predict returned HTTP 200 with a numerical prediction; and the registered champion model was loaded directly inside a running container to isolate model behavior from API behavior.

This layered approach provides stronger evidence than a browser-only test because it distinguishes frontend behavior from API, container, registry, and model-artifact failures. The final demonstration therefore covers both functional application behavior and the underlying MLOps infrastructure.

8.1 API Validation

Final validation returned HTTP 200 from /health with the champion model and HTTP 200 from /predict with a numerical predicted price. The deployed model was also executed directly inside the running Flask pod, confirming that inference was performed by the registered MLflow artifact.

8.2 Model Behavior Validation

Initial API tests using unsupported categorical values produced an identical output. Investigation showed that the pipeline uses OneHotEncoder(handle_unknown="ignore"), and the manually entered categories were outside the learned vocabulary. Subsequent tests using categories and routes present in the training data produced distinct prices, confirming that the model responds to supported inputs.

Scenario	Expected	Observed
Florianopolis (SC) → Aracaju (SE), firstClass, CloudFy	Successful prediction	Distinct price returned
Brasilia (DF) → Salvador (BH), economic, FlyingDrops	Successful prediction	Distinct price returned
Flask /health	HTTP 200	HTTP 200
MLflow champion load	READY / loadable	Successfully loaded

9. Demonstration Evidence

The following screenshots were captured during final validation and are suitable as evidence for the submission.



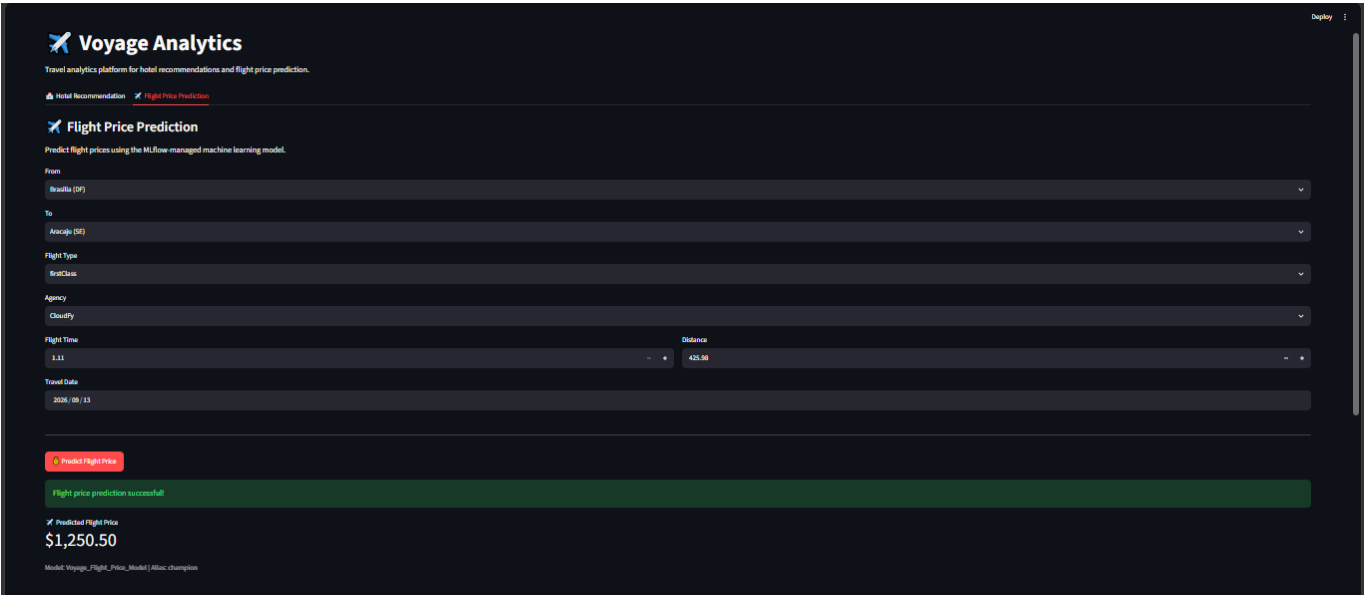


Figure 6. Flight Price Prediction Streamlit interface using dataset-supported inputs.

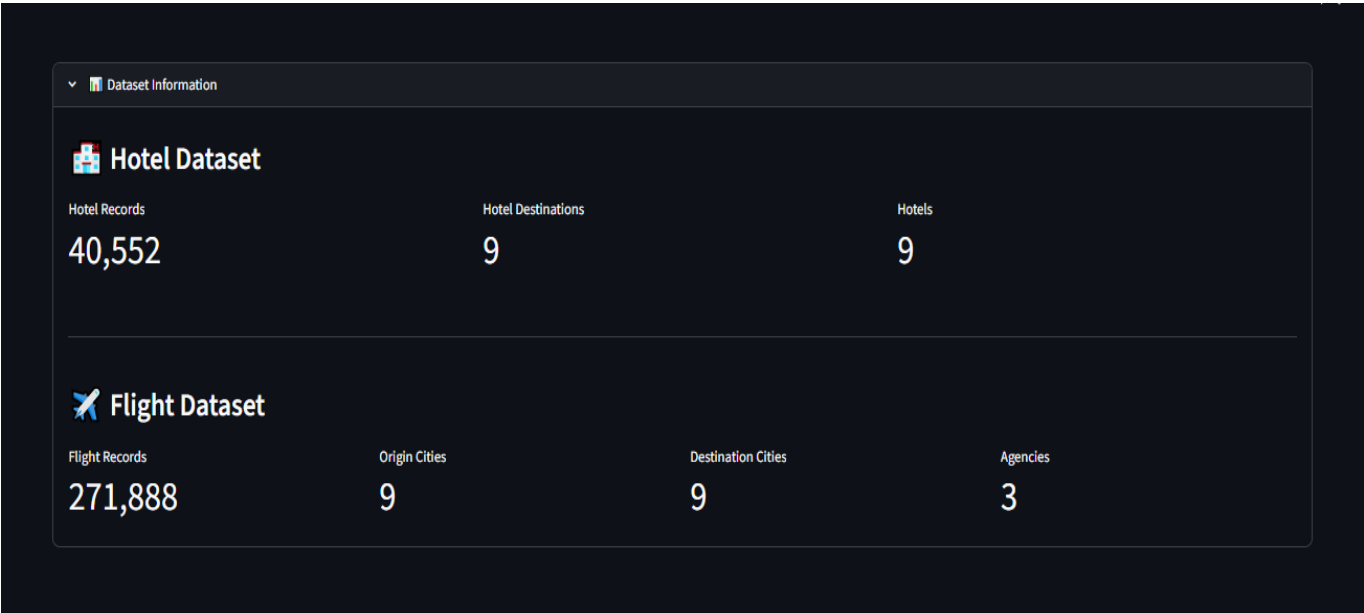


Figure 7. Dataset Information showing hotel and flight record counts.

10. Challenges and Resolutions

Additional infrastructure evidence captured during final validation:

```
(.venv) PS C:\Users\VINAY\Desktop\Labmentix Projects\Travel_MLOps_Major_Project> mlflow ui --host 0.0.0.0 --port 5000 --allowed-hosts "localhost,127.0.0.1:*,host.docker.internal:5000"
[MLflow] Security middleware enabled. Allowed hosts: localhost:*, 127.0.0.1:*, host.docker.internal:5000.
2026/09/13 04:26:31 WARNING mlflow.server: MLflow job execution requirements not met (MLflow job backend does not support Windows system.).
will start without job execution support. Errors will be surfaced at job invocation time.
2026/09/13 04:26:32 INFO:      Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)
2026/09/13 04:26:32 INFO:      Started parent process [12948]
C:\Users\VINAY\Desktop\Labmentix Projects\Travel_MLOps_Major_Project\.venv\Lib\site-packages\mlflow\server\fastapi_app.py:19: StarletteDeprecationWarning: starlette.middleware.wsgi is deprecated and will be removed in a future release. Please refer to https://github.com/abersheeran/a2wsgi as a replacement.
```

Figure 8. MLflow model registry and production champion alias.

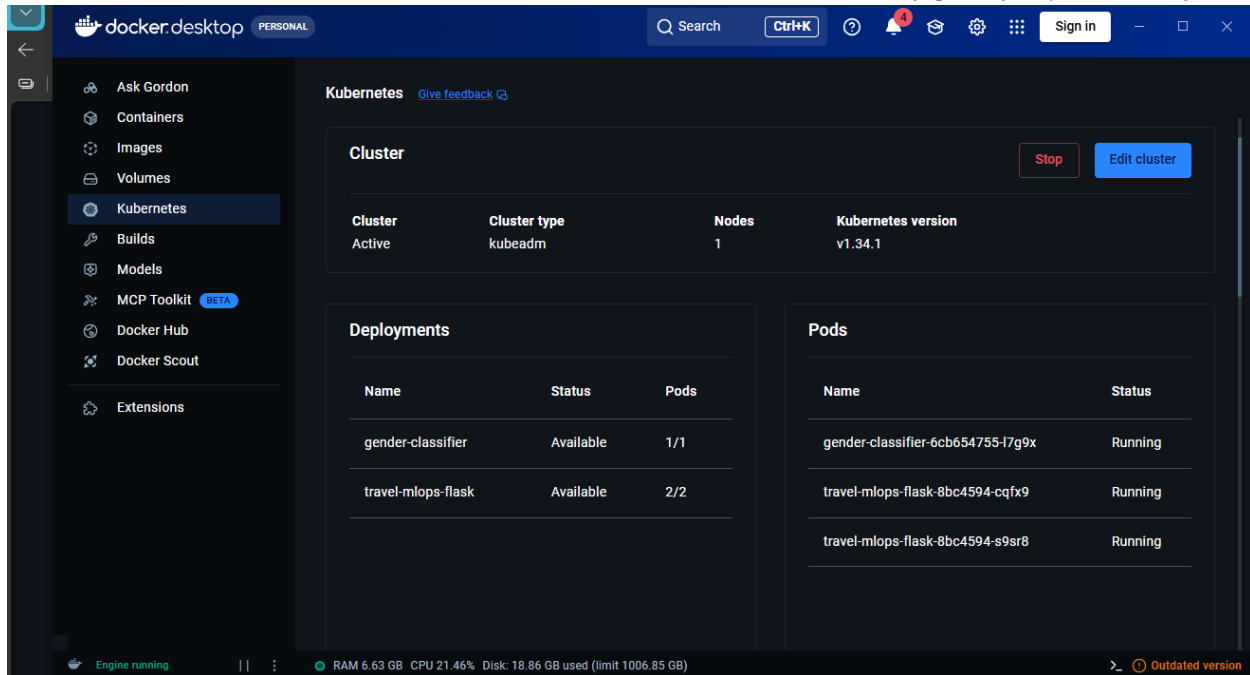


Figure 9. Docker Desktop Kubernetes deployment showing two healthy travel-mlops-flask replicas.

- MLflow Host-header security: the Flask container initially received HTTP 403 responses from MLflow because host.docker.internal:5000 was rejected. The MLflow server was restarted with an explicit allowed-host configuration.
- Kubernetes instability during development: Docker Desktop Kubernetes temporarily returned EOF/TLS handshake errors. The control plane was restored and the deployment was rolled out successfully.
- Probe sensitivity during heavy load: startup/readiness/liveness probes were tuned to tolerate longer application startup time.
- Unsupported flight categories during manual tests: the UI was revised to populate flight values from flights.csv and to constrain destinations by origin, preventing invalid categorical combinations.
- Duplicate Streamlit output: an older validation block remained in the flight tab. It was removed so the prediction result is rendered only once.

11. Limitations

- The current local demonstration environment depends on Docker Desktop, Kubernetes, MLflow, and a local port-forward.
- The model supports the categorical vocabulary represented in the training dataset; unseen categorical values are encoded as unknown by the preprocessing pipeline.
- The hotel recommendation module is rule/filter based rather than a learned recommender model.
- The current solution is primarily a local demonstration and has not been positioned as a public cloud production service.

12. Future Enhancements

- Add automated model retraining and scheduled evaluation through Airflow.
- Add richer MLflow model metadata, input signatures, tags, and promotion gates.
- Add automated API tests and CI quality gates in Jenkins.

- Add cloud deployment and managed observability/monitoring.
- Improve hotel recommendation ranking with a learned recommendation model.
- Add route-aware validation and richer flight search constraints in the UI.

13. Conclusion

Voyage Analytics demonstrates a complete applied MLOps workflow for a travel use case. The project integrates data preparation, model development, experiment and model management with MLflow, automated workflow execution with Airflow, REST inference with Flask, containerization with Docker, orchestration and scaling with Kubernetes, and an interactive Streamlit front end. The final validation confirms that the major components operate together and that the application provides a strong basis for a live project demonstration.

14. Demonstration Checklist

1. Start MLflow and confirm the Voyage_Flight_Price_Model champion model is available.
2. Confirm Kubernetes node and travel-mlops-flask pods are healthy.
3. Start kubectl port-forward on port 5001.
4. Start the Streamlit application.
5. Demonstrate one hotel recommendation scenario and change the destination.
6. Demonstrate one flight prediction using a dataset-supported route and change the route/flight type.
7. Show the MLflow model/alias and the Airflow DAG as MLOps evidence.
8. Conclude with the architecture and key technologies.

Appendix A. Technical Component Summary

Layer	Technology	Purpose
Presentation	Streamlit	Interactive hotel recommendation and flight prediction dashboard
API / Serving	Flask	REST health and prediction endpoints
Model Management	MLflow	Tracking, registry, versioning, champion alias
Workflow Automation	Apache Airflow	Data validation, training, verification
Containerization	Docker	Reproducible inference-service packaging
Orchestration	Kubernetes	Deployment, Service, probes, replicas
Autoscaling	HPA	Horizontal scaling within configured bounds
Modeling	Pandas + scikit-learn + LightGBM	Feature preparation and regression pipeline

Appendix B. Key Commands Used for Demonstration

```
mlflow ui --host 0.0.0.0 --port 5000 --allowed-hosts  
"localhost:*,127.0.0.1:*,host.docker.internal:5000"  
  
kubectl get nodes  
  
kubectl get pods -l app=travel-mlops-flask  
  
kubectl get hpa travel-mlops-flask  
  
kubectl port-forward svc/travel-mlops-flask 5001:5001  
  
streamlit run streamlit_app.py
```